Name: _____

Roll Number: _____

Course Name: PF 2024

Exam Type: PRACTICE MidTerm 2

(Theory)

Instructions/Notes:

- Solve the exam on this question paper. Rough sheets can be used but will not be attached or marked.
- In the programming problems, you are free to use the standard C++ library functions from iostream, cstring etc. where required. 😛

MARKS Distribution

Q0	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Total
10	10	10	15	15	10	10	10	10	30	20	25	175

Question 0 (15 Marks)

Write a program which takes two sets as inputs, along with their sizes and prints their cartesian product. You may safely assume that the Size will not be greater than 100.

Sample Input

Size = 3, A [ ] = {1,2,3}
Size = 4, B [ ] = {8,9,10,11}

Sample Output

AxB [ ] has Size of 5x4 = { (1,8), (1,9), (1, 10),(1,11),(2,8), (2,9), (2, 10),(2,11), (3,8), (3,9), (3, 10),(3,11) }

```
#include <iostream>
#include<fstream>
using namespace std;
void fillarr(int arr[],int cap,int &size) {}
void printcart(int arr1[], int size1,int arr2[],int size2) {}

int main()
{
      const int cap = 100;
       int  size1 = 0, size2 = 0;
       int arr1[cap];
       int arr2[cap];
       fillarr(arr1,cap,size1);
       fillarr(arr2, cap, size2);

       printcart(arr1, size1, arr2, size2);
}
```

Question 1 - Self Transpose of a Matrix (10 Marks)

Part A

```
// If there is anything to write, write here

void SelfTranspose(int Matrix[____][_____], int &Rows, int &Cols) {}
void ReadInput(int Matrix[____][___], int& Rows, int& Cols, const char filename[]) {}
void PrintMatrix(int Matrix[___][___], int Rows, int Cols)
{
        for (int i = 0; i < Rows; i++)
        {
                for (int j = 0; j < Cols; j++)
                {
                        cout<<Matrix[i][j]<<"\t";
                }
                cout << endl;
        }
}
int main()
{
   int  Matrix[maxRows][maxCols], Rows, Cols;
   char FileName [100];
   cout<<"Enter the file Name: ";
   cin>>FileName;
   ReadInput(Matrix, Rows, Cols, FileName);
   cout << "Matrix After Reading from File and before Transpose...\n";
   PrintMatrix(Matrix,Rows,Cols);
   SelfTranspose(Matrix, Rows, Cols);
   cout << "Matrix After Transpose...\n";
   PrintMatrix(Matrix,Rows,Cols);
}
```

Question 2 (15+15 Marks)

Part A

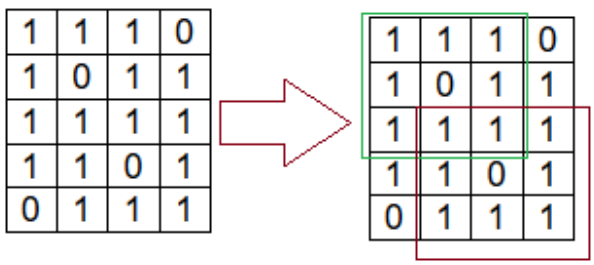
A cell is the basic structural, functional, and biological unit of all known organisms. The cell membrane, the boundary of a cell, is a biological membrane that separates the interior of all cells from the outside environment. Cell viability and its proper functioning can be assessed by measuring the amount of live and dead cells in a total cell sample. Live cells have intact cell membranes and contain properly functioning enzymes.

You are given a black and white image of an organism's cells sample, which is stored in a matrix ***data*** of dimensions m x n (allocated dynamically by taking input from the file - **you don't need to worry about its loading while writing this function; it is already there with you**). That is, it has m rows, n columns. Matrix *data* contains only 0s and 1s. **0 represents a black pixel and 1 represents white pixel**. We are interested in **counting the number of k x k blocks** (sub-matrices) in *data* that have all the white pixels over the boundary (where the boundary is actually representing the cell membrane of a cell). We call such k x k blocks "**live cells**" having intact cell membranes. **Write a C++ function that returns the count of "live cells" in data.**

You may use the function header: **int countLiveCells(int data[___][___], int m, int n, int k)**

For further clarity consider the following example: Here matrix *data* has dimensions m = 5, n=4, and let's take k=3. Take each 3 x 3 sub-matrix (this example has a total of 6 such matrices), and check if there is 3x3 block containing all boundary cells as whites.

In the example there are 2 "*mostly white blocks*" of size 3x3 with their top-left corners at (0, 0), (2, 2) respectively.




```
#include <iostream>
#include<fstream>
using namespace std;
int countLiveCells(int **data, int m, int n, int k) {}
bool iscelllive(int** data, int m, int n, int ri, int ci, int k) {}
void loadImage(int data[____][___],int &m,int &n){
       ifstream Rdr("Cells.txt");
       Rdr >> m >> n;
       // Load image here
}
int main(){
       int data[____][____], m, n,k;
       cout << "Enter the Window size: "; cin >> k;
       loadImage(data, m, n);
       int wc = countLiveCells(data, m, n,k);
       cout << "We have found " << wc << " live cells...";
       return 0;
}
```

Part B

Suppose in part (a) above we also asked you to report the top left coordinates (row number and col number of the top left cell) of each and every *live cell* in *data*. Write down the changed function header that would be required to accomplish this task. Particularly, what new parameters may be required? Explain the purpose of these parameters in two lines. And recode the program fulfilling the above task **(HINT: You may need to take positions).**

```
#include <iostream>
#include<fstream>
using namespace std;
#define max 100
int countLiveCells(int data[___][___], int m, int n, int k) {}
int main()
{
	int data[][], m, n,k;
	int pr[max], pc[max];
	cout << "Enter the Window size: ";
	cin >> k;
	loadImage(data, m, n);
	int wc = countLiveCells(data, m, n,k, _________);
	cout << "We have found " << wc << " mostly white blocks...";
	cout << "The top-left co-ordinates of live cells: " << endl;
	for (int i = 0; i < wc; i++)
	{
		cout << "(" << pr[i]<< " , " << pc[i]<< "), ";
	}
}
```

Question 3: "The sieve of Eratosthenes" (25 points)

The Greek mathematician, Eratosthenes, came up with the following algorithm to quickly find *all* prime numbers between 2 and a given number n. How does it work?

Look at this animation:
https://i.stack.imgur.com/4lsjZ.gif

Start with d = 2

1) Cross out all multiples of d greater than d and less than or equal to n.
2) Set d equal to the next uncrossed number.
3) If d is greater than square_root of n, STOP.
   ELSE repeat from step (ii)
4) Report all uncrossed elements between 2 and n as primes.

Write a function called **sieveOfEratosthenes** that accepts a number n and returns all the primes between 2 and n as an array (passed as parameter). It should also return the count of these prime numbers. Write its test main too.